SOFTWARE ENGINEERING · CHAPTER 03

Hibernate Framework and Spring Data JPA

Hibernate and Jakarta Persistence (JPA) map Java objects to relational database tables, reducing the need for repetitive persistence code. Spring Data JPA builds repository abstractions on top of JPA.

Hibernate ORM 7.4 | Jakarta Persistence 3.2 | Spring Boot 4.1 · Spring Data JPA | JDK 25 LTS | Flyway · H2 · PostgreSQL

AGENDA

What we will cover

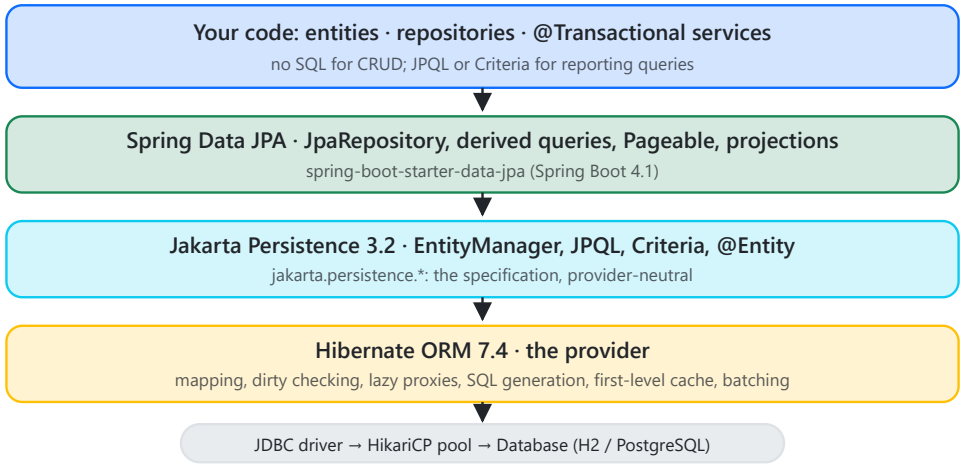
1. Introduction: from JDBC to JPA, Hibernate and Spring Data
2. Entities, identifiers, and persistence contexts
3. Object-relational mapping and associations
4. Fetch strategies, cascading, and orphan removal
5. JPQL, native queries, and the Criteria API
6. Repository interfaces and derived queries
7. Entity state transitions and dirty checking
8. Transactions and optimistic or pessimistic locking
9. Pagination, sorting, and projections
10. N+1 queries and performance tuning
11. Database migrations and schema management
12. Chapter Quiz
13. Lab-03

Click any item to jump directly to that topic.

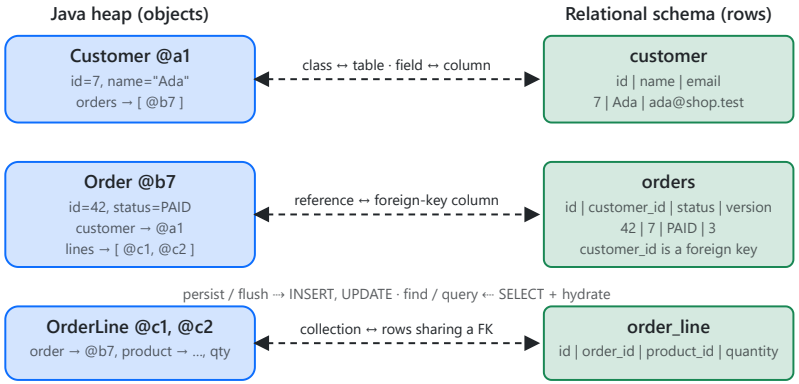
From JDBC to JPA: why an object-relational mapper?

- **The impedance mismatch**: objects have references, collections, inheritance and identity by reference; tables have rows, foreign keys and identity by primary key. In Chapter 02 we bridged the gap by hand with `JdbcClient` and row mappers.
- **Jakarta Persistence 3.2** (`jakarta.persistence.*`, part of Jakarta EE 11) is the *specification*: annotations, `EntityManager`, JPQL, Criteria API.
- **Hibernate ORM 7.4** is the *provider*: it generates SQL, tracks changes (dirty checking), loads lazily and caches identity.
- **Spring Data JPA** adds repositories: you declare an interface, Spring implements CRUD, paging and query methods at start-up.
- The price: you must still understand the SQL that is generated. This chapter shows how to read it.

Layer	You write	You get
Spring Data JPA	interfaces, method names	CRUD, paging, proxies
JPA API	entities, JPQL, Criteria	portable persistence
Hibernate	properties, a few annotations	SQL, caching, batching



Entities: Java classes that map to tables



- An entity needs `@Entity`, an `@Id`, a no-arg constructor (may be `protected`) and must not be `final` (Hibernate subclasses it for lazy proxies).
- Field access is the default when `@Id` sits on a field: no setter for `id` is needed.
- Anything not annotated is mapped by convention; `@Transient` excludes a field.

```java
import jakarta.persistence.*;

@Entity
@Table(name = "product")
public class Product {

    @Id
    @GeneratedValue(strategy = GenerationType.SEQUENCE,
                    generator = "product_seq")
    @SequenceGenerator(name = "product_seq", allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true, length = 20)
    private String sku;

    @Column(nullable = false, length = 120)
    private String name;

    @Embedded
    private Money price;                // → amount + currency columns

    @Enumerated(EnumType.STRING)        // never ORDINAL
    @Column(nullable = false, length = 20)
    private Category category;

    private int stock;                  // column "stock" by convention

    protected Product() { }             // required by JPA, not public API

    public Product(String sku, String name, Money price, Category cat) {
        this.sku = sku; this.name = name;
        this.price = price; this.category = cat;
    }

    public Long getId() { return id; }   // no setId(): the DB owns it
    public int getStock() { return stock; }
    public void setStock(int stock) { this.stock = stock; }
    // name, sku, price, category accessors …
}
```

Identifiers, `@Embeddable` value objects and enums

```java
@Embeddable                          // value object: no identity, no table
public record Money(
        @Column(name = "amount", precision = 12, scale = 2)
        BigDecimal amount,
        @Column(name = "currency", length = 3)
        String currency) {

    public static Money eur(String v) {
        return new Money(new BigDecimal(v), "EUR");
    }
    public Money times(int qty) {
        var total = amount.multiply(BigDecimal.valueOf(qty));
        return new Money(total, currency);
    }
}

@Embeddable
public record Address(String street, String city, String zip) { }

@Entity
public class Customer {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE,
                        generator = "customer_seq")
    @SequenceGenerator(name = "customer_seq", allocationSize = 50)
    private Long id;

    @Column(nullable = false, unique = true, length = 160)
    private String email;

    @Embedded
    private Address billing;             // street, city, zip

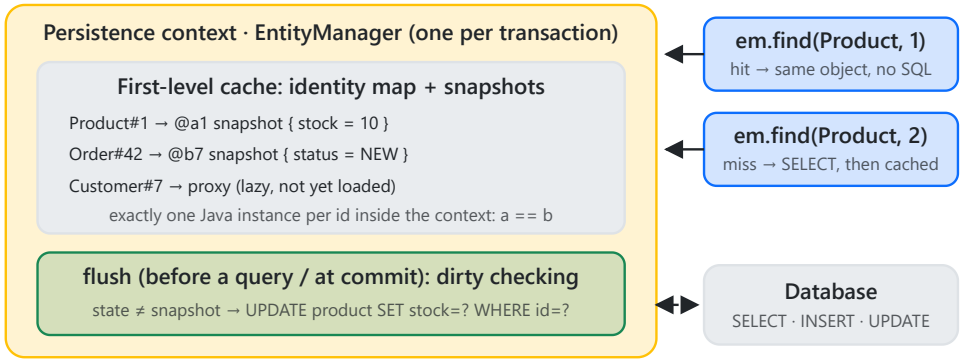
    @Embedded                            // same type twice: rename columns
    @AttributeOverride(name = "street", column = @Column(name = "ship_street"))
    @AttributeOverride(name = "city",   column = @Column(name = "ship_city"))
    @AttributeOverride(name = "zip",    column = @Column(name = "ship_zip"))
    private Address shipping;
    // …
}
```

`GenerationType`	How	Verdict
`SEQUENCE`	DB sequence; `allocationSize=50` reserves 50 ids per round trip	default choice, batching works
`IDENTITY`	auto-increment column, id known only after INSERT	disables JDBC batching (MySQL legacy)
`UUID`	generated in Java (JPA 3.1+)	distributed systems; 16 bytes, random order
`TABLE`	extra table with a counter row	slow, locks; avoid
`AUTO`	provider picks (Hibernate: SEQUENCE)	fine, but be explicit

- Records can be `@Embeddable` since Hibernate 6.2 / Jakarta Persistence 3.2: immutable, `equals` by value, exactly what a value object needs.
- `@Enumerated(EnumType.STRING)` stores the constant name. `ORDINAL` stores the position: reordering the enum silently corrupts data.
- Natural keys (`sku`, `email`) get a `unique` constraint, but the primary key stays a surrogate `Long`.

⚠ The sequence in the database must match `allocationSize` (`INCREMENT BY 50`). Hibernate 7 validates this at start-up and fails with "*Sequence increment size mismatch*".

The persistence context: a first-level cache with a memory



At commit the context closes: entities become detached. An optional second-level cache spans transactions.

```java
@Repository
public class ProductDao {
    @PersistenceContext          // proxy to the EntityManager of the
    private EntityManager em;    // current transaction (thread-bound)

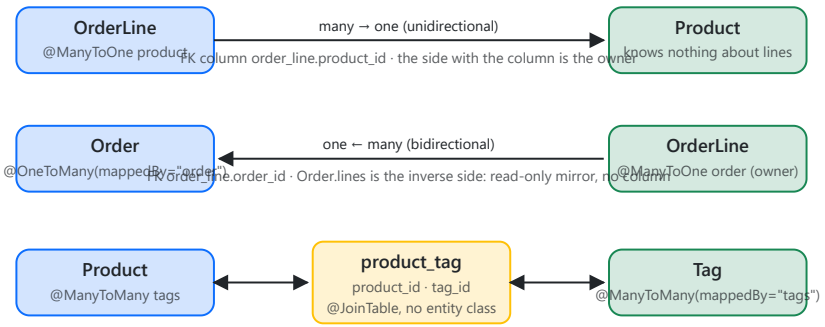
    @Transactional
    public void demo() {
        Product a = em.find(Product.class, 1L);  // SELECT … WHERE id=1
        Product b = em.find(Product.class, 1L);  // cache hit: no SQL
        assert a == b;                           // same instance

        a.setStock(a.getStock() - 1);   // no SQL yet; snapshot differs
        em.contains(a);                 // true: a is managed

        Customer ref = em.getReference(Customer.class, 7L);
        // a proxy with only the id set; no SELECT until a field is read
    }   // commit → flush → UPDATE product SET stock=? WHERE id=1
}
```

- **Scope**: with Spring, the persistence context lives exactly as long as the `@Transactional` method (transaction-scoped). No transaction, no context: every repository call gets its own short-lived one.
- **Repeatable reads for free**: the same id always yields the same instance within a context.
- **Write-behind**: SQL is delayed until flush, so Hibernate can reorder and batch statements.
- `spring.jpa.open-in-view=false` keeps the context out of the web layer; the default (`true`) is a well-known trap (Topic 9).

Associations: references and collections become foreign keys



many ↔ many: join table. Promote it to an entity when the link needs attributes (added_at, position).

Annotation	Column / table
`@ManyToOne` + `@JoinColumn`	FK on this entity's table
`@OneToMany(mappedBy)`	nothing: mirrors the FK
`@OneToMany` without `mappedBy`	join table (usually unwanted)
`@OneToOne`	FK + unique, or shared PK with `@MapsId`
`@ManyToMany` + `@JoinTable`	join table with two FKs

```java
@Entity
@Table(name = "orders")                 // ORDER is an SQL keyword
public class Order {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE,
                        generator = "orders_seq")
    @SequenceGenerator(name = "orders_seq", allocationSize = 50)
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "customer_id")   // FK column on orders
    private Customer customer;

    @Enumerated(EnumType.STRING)
    private OrderStatus status = OrderStatus.NEW;

    private Instant placedAt = Instant.now();

    @OneToMany(mappedBy = "order",      // OrderLine.order owns the FK
               cascade = CascadeType.ALL, orphanRemoval = true)
    private List<OrderLine> lines = new ArrayList<>();

    @Version
    private long version;
    // constructor, accessors, helpers (next slide) …
}

@Entity
@Table(name = "order_line")
public class OrderLine {
    @Id @GeneratedValue(strategy = GenerationType.SEQUENCE,
                        generator = "order_line_seq")
    private Long id;                    // generator declared as above

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "order_id")
    private Order order;                // owning side: has the FK column

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    private Product product;            // FK column defaults to product_id

    private int quantity;

    @Embedded
    private Money unitPrice;            // price copied at order time
}
```

Keeping both sides in sync, many-to-many, and entity identity

```java
// Order (aggregate root): the only place that touches the collection
public void addLine(Product product, int qty) {
    OrderLine line = new OrderLine(this, product, qty);  // sets line.order
    lines.add(line);                    // cascade PERSIST inserts it
}
public void removeLine(OrderLine line) {
    lines.remove(line);                 // orphanRemoval deletes the row
    line.setOrder(null);
}
public List<OrderLine> getLines() {
    return Collections.unmodifiableList(lines);
}

// Product ⇔ Tag: many-to-many through a join table
@ManyToMany
@JoinTable(name = "product_tag",
           joinColumns = @JoinColumn(name = "product_id"),
           inverseJoinColumns = @JoinColumn(name = "tag_id"))
private Set<Tag> tags = new HashSet<>();   // Set, not List (no "bag")

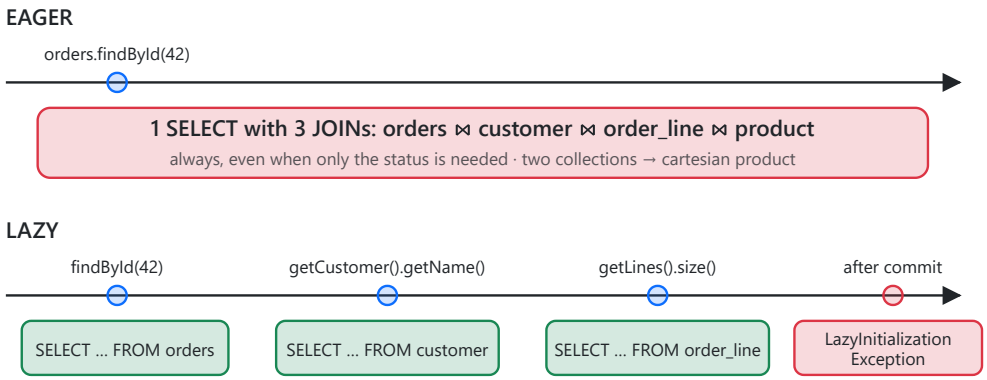
// Identity: stable before and after the id is assigned
@Override public boolean equals(Object o) {
    return this == o
        || (o instanceof Order other && id != null
            && id.equals(other.getId()));   // getId(): works on proxies
}
@Override public int hashCode() { return Order.class.hashCode(); }

@Override public String toString() {          // never touch associations
    return "Order[id=%d, status=%s]".formatted(id, status);
}
```

- **Only the owning side is written.** Adding a line to `order.lines` without setting `line.order` inserts a row with `order_id = NULL` (or fails on `NOT NULL`). Helper methods keep both sides consistent.
- **Expose the collection read-only**; callers go through `addLine` / `removeLine`.
- **Set vs List**: a `List` without `@OrderColumn` is a *bag*; Hibernate refuses to fetch two bags at once (`MultipleBagFetchException`) and re-inserts the whole collection on some changes.
- **equals/hashCode**: a new entity has `id == null`; after `persist` it has one. A hash based on the id (or on all fields) changes, so the entity is lost in a `HashSet`. Compare by id when present, keep a constant hash.

Pitfall	Symptom
Lombok `@Data` on entities	`toString` loads or recurses through associations (StackOverflowError, LazyInitializationException)
`hashCode` from all fields	`Set.contains` fails after edits or persist
Missing `mappedBy`	extra join table `orders_order_line` appears
Two eager `List` collections	`MultipleBagFetchException` at start-up

Fetch strategies, cascading and orphan removal



s load on first access, only while the persistence context is open. Fetch what a use case needs with JOIN FETCH or an EntityGraph (To

Cascade	Parent operation propagates to children
`PERSIST`	`persist(order)` also inserts new lines
`MERGE`	`merge(order)` merges detached lines too
`REMOVE`	`remove(order)` deletes its lines
`REFRESH`, `DETACH`	reload / detach the graph
`ALL`	all of the above; right for composition (Order → OrderLine), wrong for shared entities (Product)
`orphanRemoval=true`	a child removed from the collection is deleted, not just unlinked

```
// Defaults: @ManyToOne, @OneToOne = EAGER; @OneToMany, @ManyToMany = LAZY
@ManyToOne(fetch = FetchType.LAZY)         // always override to LAZY
@JoinColumn(name = "customer_id")
private Customer customer;

@OneToMany(mappedBy = "order",
           cascade = { CascadeType.PERSIST, CascadeType.MERGE,
                       CascadeType.REMOVE },
           orphanRemoval = true)
private List<OrderLine> lines = new ArrayList<>();

@ManyToOne(fetch = FetchType.LAZY)         // no cascade: Product is shared
private Product product;

// The transaction is the boundary of lazy loading
@Transactional(readOnly = true)
public OrderView view(Long id) {
    Order o = orders.findById(id).orElseThrow();
    // touching proxies here runs the extra SELECTs, still inside the tx
    return OrderView.from(o, o.getCustomer(), o.getLines());
}

// Outside a transaction (open-in-view=false):
Order o = orders.findById(42L).orElseThrow();   // own short tx, closed
o.getLines().size();   // → LazyInitializationException
```

Rule of thumb: map everything LAZY, decide per query what to fetch. `CascadeType.REMOVE` deletes children when the parent is removed; `orphanRemoval` also deletes a child that is merely taken out of the collection.

JPQL: queries over entities, not tables

```
public interface OrderRepository extends JpaRepository<Order, Long> {

    // JPQL: entity names and attributes; implicit join via o.customer
    @Query("""
        select distinct o from Order o
          join fetch o.lines l
          join fetch l.product
         where o.customer.email = :email
           and o.status = :status
        order by o.placedAt desc
        """)
    List<Order> findWithLines(@Param("email") String email,
                              @Param("status") OrderStatus status);

    // aggregate / scalar result
    @Query("""
        select o.status, count(o) from Order o
         where o.placedAt >= :since group by o.status
        """)
    List<Object[]> countByStatusSince(@Param("since") Instant since);

    // native SQL for vendor features (window functions, hints, CTEs)
    @Query(value = """
        SELECT o.* FROM orders o
         WHERE o.placed_at >= :since
         ORDER BY o.placed_at DESC
         LIMIT :n
        """, nativeQuery = true)
    List<Order> latest(@Param("since") Instant since, @Param("n") int n);
}
```

JPQL	SQL
`from Order o` (entity name)	`FROM orders o` (table)
`o.customer.email` (path)	explicit `JOIN customer c ON …`
`join fetch o.lines`	join + hydrate the collection (no N+1)
`:email` named parameter	`?` bound safely, never concatenated
returns managed entities or DTOs	returns rows
portable across databases	vendor dialect

```
// Programmatic, with the EntityManager
List<Order> paid = em.createQuery(
        "select o from Order o where o.status = :s",
        Order.class)
    .setParameter("s", OrderStatus.PAID)
    .setMaxResults(20)
    .getResultList();
```

- Spring Data's `@Query` runs JPQL by default; `nativeQuery = true` switches to SQL (result mapped to the entity or to a projection).
- Named parameters work without `@Param` when the compiler keeps parameter names (`-parameters`, on by default in Spring Boot builds).
- Hibernate 7 also accepts HQL extensions: `from Order` without `select`, `limit`, `with` CTEs, `ilike`.

Type-safe dynamic queries: Criteria API and `Specification`

```
public interface ProductRepository
        extends JpaRepository<Product, Long>,
                JpaSpecificationExecutor<Product> { }

// Spring Data Specification = one Criteria predicate, composable
public final class ProductSpecs {
    static Specification<Product> nameContains(String q) {
        return (root, query, cb) -> cb.like(
                cb.lower(root.<String>get("name")),
                "%" + q.toLowerCase() + "%");
    }
    static Specification<Product> inCategory(Category c) {
        return (root, query, cb) -> cb.equal(root.get("category"), c);
    }
    static Specification<Product> cheaperThan(BigDecimal max) {
        return (root, query, cb) ->
            cb.le(root.<Money>get("price").<BigDecimal>get("amount"), max);
    }
}

// dynamic filter: a null specification means "no constraint"
Specification<Product> spec = Specification.allOf(
        q   == null ? null : ProductSpecs.nameContains(q),
        cat == null ? null : ProductSpecs.inCategory(cat),
        max == null ? null : ProductSpecs.cheaperThan(max));
Page<Product> page = products.findAll(spec, pageable);
```

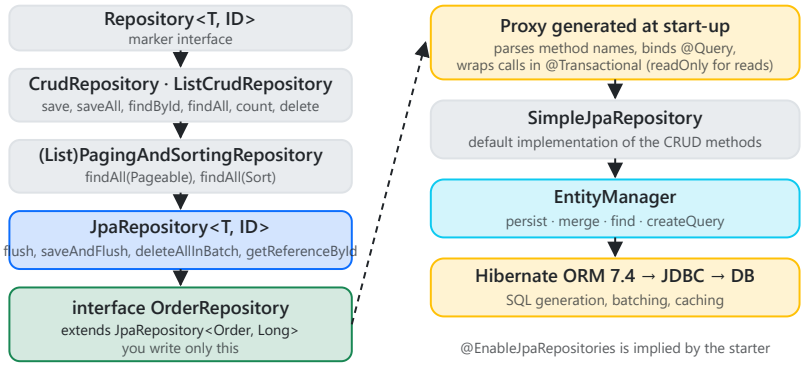
```
// Plain Criteria API (jakarta.persistence.criteria)
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Product> q = cb.createQuery(Product.class);
Root<Product> p = q.from(Product.class);
q.select(p)
 .where(cb.equal(p.get("category"), Category.BOOKS))
 .orderBy(cb.asc(p.get("name")));
List<Product> books = em.createQuery(q).getResultList();
```

```
// EntityGraph: a fetch plan attached to a repository method
@EntityGraph(attributePaths = {"lines", "lines.product"})
Optional<Order> findWithGraphById(Long id);
```

```
// Hibernate 7: SelectionQuery, typed, cannot be misused
// as an update (Session = em.unwrap(Session.class))
List<Product> low = session
    .createSelectionQuery("from Product where stock < :min",
                          Product.class)
    .setParameter("min", 5)
    .getResultList();
```

- Strings like `"name"` are checked only at runtime. The **static metamodel** (`Product_.name`, generated by `org.hibernate.orm:hibernate-processor`) makes them compile-time safe.
- Criteria shines for search screens with optional filters; for fixed queries JPQL reads better.

Spring Data JPA: declare the interface, get the implementation




```java
public interface OrderRepository extends JpaRepository<Order, Long> {

    // derived queries: parsed from the method name at start-up
    List<Order> findByCustomerEmailAndStatusOrderByPlacedAtDesc(
            String email, OrderStatus status);

    Optional<Order> findFirstByCustomerIdOrderByPlacedAtDesc(Long cid);

    long countByStatus(OrderStatus status);

    boolean existsByCustomerIdAndStatus(Long customerId, OrderStatus s);

    List<Order> findTop5ByStatusOrderByPlacedAtDesc(OrderStatus status);

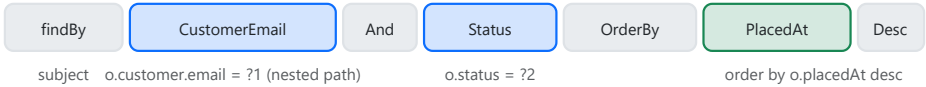
    Stream<Order> streamByPlacedAtBefore(Instant t);   // needs an open tx
}

@Service
public class OrderService {
    private final OrderRepository orders;
    private final CustomerRepository customers;
    private final ProductRepository products;
    // constructor injection …

    @Transactional
    public Order place(Long customerId, Map<Long, Integer> items) {
        Customer c = customers.getReferenceById(customerId); // no SELECT
        Order o = new Order(c);
        items.forEach((pid, qty) -> {
            Product p = products.findById(pid).orElseThrow();
            o.addLine(p, qty);
        });
        return orders.save(o);      // persist + cascade to the lines
    }

    @Transactional(readOnly = true)
    public List<Order> paidOrdersOf(String email) {
        return orders.findByCustomerEmailAndStatusOrderByPlacedAtDesc(
                email, OrderStatus.PAID);
    }
}
```

Method-name grammar, `@Modifying` and custom fragments



Validated when the context starts: a typo in a property name fails the boot, not a production request.

```
public interface ProductRepository extends JpaRepository<Product, Long>,
                                           ProductRepositoryCustom {

    // bulk update: no entities loaded; context cleared afterwards
    @Modifying(clearAutomatically = true)
    @Query("""
        update Product p set p.stock = p.stock - :qty
         where p.id = :id and p.stock >= :qty
        """)
    int reserve(@Param("id") Long id, @Param("qty") int qty);

    @Modifying
    @Query("delete from Product p where p.stock = 0 and p.category = :c")
    int purge(@Param("c") Category c);

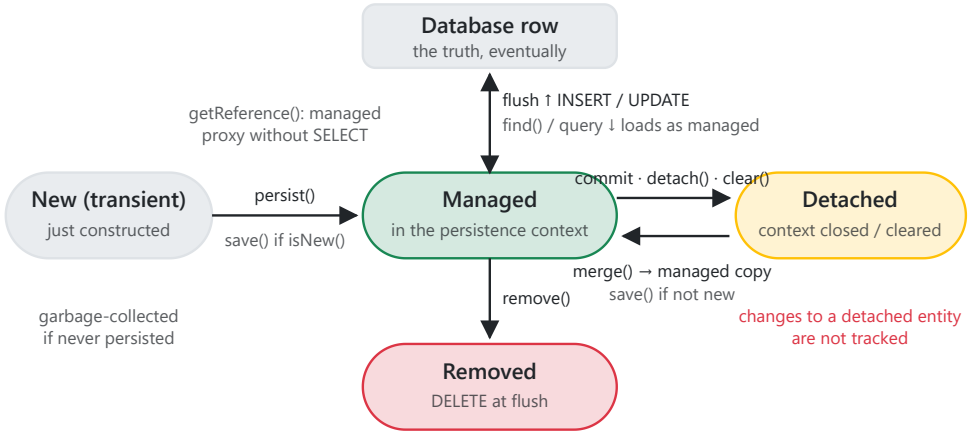
    // nested embeddable property + limit, all from the name
    List<Product> findTop3ByCategoryOrderByPriceAmountDesc(Category c);
}

// custom fragment: hand-written code next to the derived methods
interface ProductRepositoryCustom {
    List<Product> search(String q, Category c, BigDecimal max);
}
class ProductRepositoryCustomImpl implements ProductRepositoryCustom {
    @PersistenceContext EntityManager em;
    @Override public List<Product> search(String q, Category c,
                                          BigDecimal max) {
        // Criteria API or Hibernate-specific code goes here
    }
}
```

Keyword	JPQL
`find…By, read…By, get…By, query…By, stream…By`	`select`
`count…By, exists…By, delete…By`	count / exists / delete
`And, Or`	logical operators
`Between, LessThan, GreaterThanEqual, After, Before`	comparisons
`Like, Containing, StartingWith, IgnoreCase`	`like` with wildcards / `upper()`
`In, IsNull, IsNotNull, True, False`	membership, null, boolean
`Top3, First, Distinct`	limit / distinct
`OrderBy…Asc/Desc`	`order by`

⚠ `@Modifying` queries bypass the persistence context: entities already loaded keep their old values unless the context is cleared. They also need a transaction (`TransactionRequiredException` otherwise).

Entity lifecycle: new → managed → detached / removed




```java
@Transactional
public void lifecycle() {
    Product p = new Product("KB-1", "Keyboard", Money.eur("49.90"),
                            Category.HARDWARE);         // NEW
    em.persist(p);          // MANAGED: id taken from the sequence
    p.setStock(10);         // tracked, still no SQL
    em.flush();             // INSERT INTO product … (one statement)

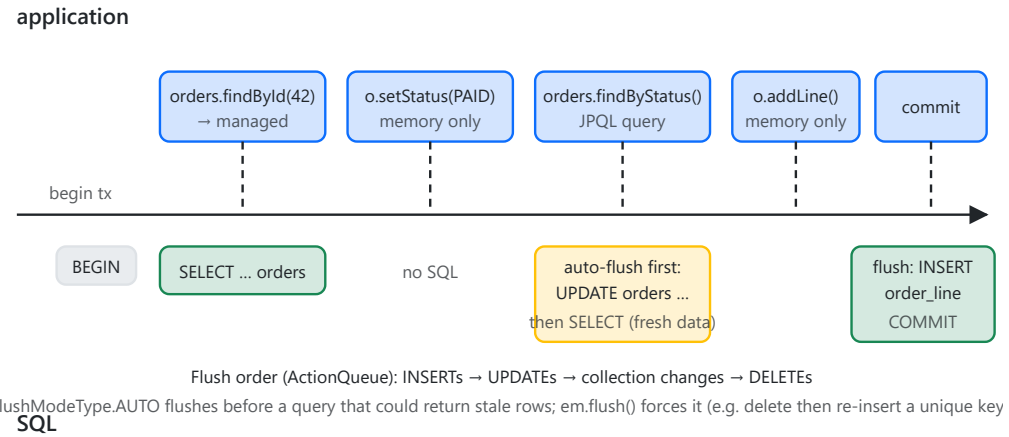
    em.detach(p);           // DETACHED
    p.setStock(5);          // not tracked: lost unless merged
    Product m = em.merge(p);// SELECT, then copy state onto a managed copy
    assert m != p;          // merge returns the managed instance

    em.remove(m);           // REMOVED: DELETE at flush / commit
}

// Spring Data: save() = persist() for new entities, merge() otherwise
Order saved = orders.save(order);   // isNew(): id == null (or @Version)
// Preferred style: load → modify inside the transaction → done.
// No save() call is needed for a managed entity (next slide).
```

Operation	Effect
`persist(e)`	schedule INSERT; e becomes managed (no copy)
`merge(e)`	load or reuse managed instance, copy e's state onto it, return it
`remove(e)`	schedule DELETE (e must be managed)
`refresh(e)`	overwrite e with database state
`detach(e)` / `clear()`	stop tracking e / everything

Dirty checking and the flush: when does SQL actually run?



Flush order (ActionQueue): INSERTs → UPDATEs → collection changes → DELETEs

FlushModeType.AUTO flushes before a query that could return stale rows; em.flush() forces it (e.g. delete then re-insert a unique key

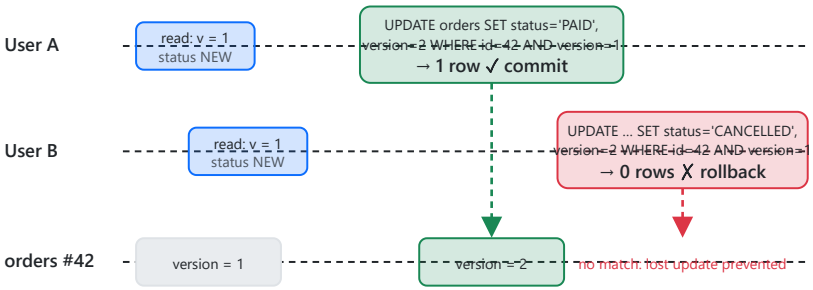
```java
// Dirty checking: no save(), no UPDATE written by you
@Transactional
public void markPaid(Long orderId) {
    Order o = orders.findById(orderId).orElseThrow();
    o.setStatus(OrderStatus.PAID);
}   // commit → UPDATE orders SET status=?, version=? WHERE id=? …

// Hibernate 7 StatelessSession: no context, no dirty checking,
// no lazy loading, no cascade. Ideal for bulk import/export.
SessionFactory sf = emf.unwrap(SessionFactory.class);
try (StatelessSession ss = sf.openStatelessSession()) {
    Transaction tx = ss.beginTransaction();
    ss.setJdbcBatchSize(500);
    ss.insertMultiple(products);      // batched INSERTs, flat memory
    tx.commit();
}

// Hibernate 7 @SoftDelete: remove() becomes UPDATE … SET deleted = true
@Entity
@SoftDelete(columnName = "deleted")   // queries filter it automatically
public class Customer { /* … */ }
```

- Hibernate compares each managed entity with its load-time snapshot; only changed rows produce an `UPDATE`, containing *all* columns unless `@DynamicUpdate`.
- Calling `repo.save(managed)` is harmless but useless: it just returns the same instance.
- A long loop that loads thousands of entities grows the context; call `flush()` + `clear()` every N items, or use a `StatelessSession`.

Transactions and optimistic locking with `@Version`



Hibernate: OptimisticLockException → Spring: ObjectOptimisticLockingFailureException

No database lock is held between read and write: cheap, scales, but the loser must retry or report.

```java
@Entity @Table(name = "orders")
public class Order {
    @Version                // long/int/Instant; never set it yourself
    private long version;    // Hibernate adds "AND version = ?" to UPDATEs
    // …
}

@Service
public class OrderService {
    @Transactional(readOnly = true)     // FlushMode MANUAL, no dirty check
    public OrderView view(Long id) { /* … */ }

    @Transactional                // REQUIRED, rollback on RuntimeException
    public void changeStatus(Long orderId, OrderStatus next) {
        Order o = orders.findById(orderId).orElseThrow();
        o.setStatus(next);
        // commit → UPDATE orders SET status=?, version=?
        //          WHERE id=? AND version=?  → 0 rows → exception
    }
}

// Retry belongs OUTSIDE the failing transaction (a different bean):
// the exception is raised at commit, when the proxy returns.
@Service
public class OrderFacade {
    @Retryable(includes = ObjectOptimisticLockingFailureException.class,
               maxAttempts = 3, delay = 50)   // Spring Framework 7 core
    public void changeStatus(Long id, OrderStatus s) {
        orderService.changeStatus(id, s);
    }
}
// @EnableResilientMethods on a @Configuration class switches it on
```

`readOnly = true` lets Hibernate skip snapshots and dirty checking, and tells the driver/pool the connection is read-only. A modification inside a read-only transaction is silently never flushed.

Pessimistic locking: `SELECT … FOR UPDATE` when conflicts are the norm

```java
public interface ProductRepository extends JpaRepository<Product, Long> {

    @Lock(LockModeType.PESSIMISTIC_WRITE)       // SELECT … FOR UPDATE
    @QueryHints(@QueryHint(name = "jakarta.persistence.lock.timeout",
                           value = "3000"))      // ms; provider-dependent
    @Query("select p from Product p where p.id = :id")
    Optional<Product> findForUpdate(@Param("id") Long id);
}

@Service
public class StockService {
    @Transactional                 // the row lock lives as long as the tx
    public void reserve(Long productId, int qty) {
        Product p = products.findForUpdate(productId)
                            .orElseThrow(() -> new NotFound(productId));
        if (p.getStock() < qty) throw new OutOfStock(productId, qty);
        p.setStock(p.getStock() - qty);     // dirty checking → UPDATE
    }                                       // COMMIT releases the lock
}

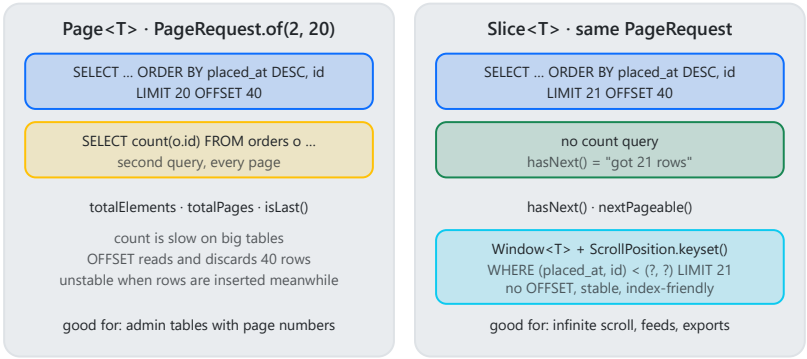
// Often better: one atomic statement, no lock held in Java
@Transactional
public void reserveAtomic(Long productId, int qty) {
    if (products.reserve(productId, qty) == 0)   // @Modifying UPDATE …
        throw new OutOfStock(productId, qty);    // WHERE stock >= :qty
}
```

	Optimistic (`@Version`)	**Pessimistic (`@Lock`)**
Mechanism	version column checked in UPDATE	database row lock until commit
Cost while idle	none	blocks other writers, risk of deadlock/timeouts
Failure mode	exception at commit, retry	waiting, `PessimisticLockException` on timeout
Use when	conflicts are rare, user-facing edits	hot rows (stock, counters, seat booking)

`LockModeType`	**Meaning**
`OPTIMISTIC`	verify version at commit even if unchanged
`OPTIMISTIC_FORCE_INCREMENT`	bump the parent version when a child changes
`PESSIMISTIC_READ`	shared lock (`FOR SHARE`)
`PESSIMISTIC_WRITE`	exclusive lock (`FOR UPDATE`)
`PESSIMISTIC_FORCE_INCREMENT`	exclusive lock + version bump

⚠ Lock in a consistent order (by id) and keep locking transactions short. A lock acquired inside `@Transactional(readOnly = true)` or outside a transaction is a bug.

Pagination and sorting: `Pageable`, `Page`, `Slice`, `Sort`




```java
public interface OrderRepository extends JpaRepository<Order, Long> {
    Page<Order>  findByStatus(OrderStatus status, Pageable pageable);
    Slice<Order> findByCustomerId(Long customerId, Pageable pageable);
    List<Order>  findByStatus(OrderStatus status, Sort sort);

    // keyset scrolling (Spring Data 3.1+): stable, no OFFSET
    Window<Order> findFirst20ByStatusOrderByPlacedAtDescIdDesc(
            OrderStatus status, ScrollPosition position);
}

@RestController
@RequestMapping("/api/orders")
public class OrderController {
    private final OrderRepository orders;
    public OrderController(OrderRepository orders) { this.orders = orders; }

    // GET /api/orders?status=PAID&page=2&size=20&sort=placedAt,desc
    @GetMapping
    public Page<OrderSummary> list(
            @RequestParam OrderStatus status,
            @PageableDefault(size = 20, sort = "placedAt",
                             direction = Sort.Direction.DESC)
            Pageable pageable) {
        return orders.findByStatus(status, pageable).map(OrderSummary::of);
    }
}

// by hand: always add a unique tie-breaker to the sort
Pageable p = PageRequest.of(2, 20,
        Sort.by(Sort.Direction.DESC, "placedAt").and(Sort.by("id")));
```

⚠ Cap the page size (`spring.data.web.pageable.max-page-size`, default 2000). A fetch join of a collection combined with `Pageable` makes Hibernate paginate *in memory* (log warning `HHH90003004`): paginate the root query, load collections with a second query or a batch size.

Projections: select only what the view needs

```java
// 1. Interface projection (closed): only these columns are selected
public interface OrderSummary {
    Long getId();
    OrderStatus getStatus();
    Instant getPlacedAt();
    CustomerName getCustomer();              // nested projection
    interface CustomerName { String getName(); }
}
List<OrderSummary> findByStatus(OrderStatus status);  // return type decides

// 2. Record (DTO) projection with a constructor expression
public record ProductView(Long id, String name, BigDecimal amount) { }

@Query("""
    select new edu.itc.shop.web.ProductView(p.id, p.name, p.price.amount)
      from Product p where p.category = :cat order by p.name
    """)
List<ProductView> viewsByCategory(@Param("cat") Category cat);

// 2b. derived query + record: Spring Data builds the constructor call
Page<ProductView> findByCategory(Category cat, Pageable page);

// 3. Dynamic projection: the caller chooses the shape
<T> List<T> findByStatus(OrderStatus status, Class<T> type);

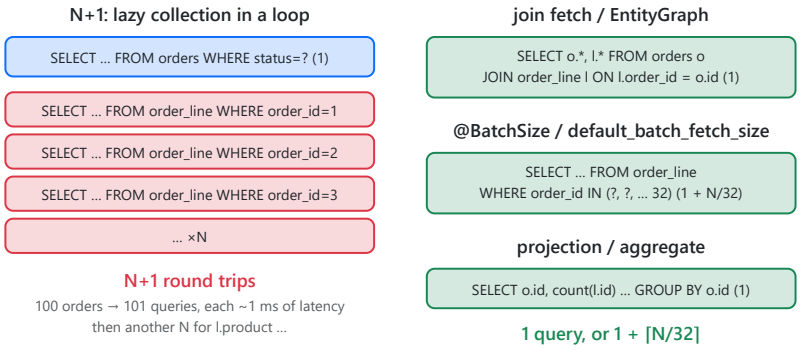
// 4. Open projection: SpEL, computed in Java, loads the whole entity
@Value("#{target.customer.name + ' (' + target.lines.size() + ')'}")
String getLabel();
```

Kind	SQL selects	Notes
Entity	all columns + lazy proxies	needed when you modify
Closed interface	listed getters only	Spring creates a proxy per row; nested paths join
Open interface (`@Value`)	whole entity	convenient, no SQL savings
Record / class DTO	constructor arguments	immutable, serialises cleanly, fastest
Dynamic `Class<T>`	depends on T	one repository method, many views

- Projections are read-only: no dirty checking, no persistence-context growth, no lazy traps. Perfect for REST responses.
- Never return entities from controllers: Jackson walks every association (LazyInitializationException, infinite recursion on bidirectional links, leaks columns).
- Since Hibernate 6.x, JPQL can omit `new` and the package when the record is imported ("implicit instantiation"); the explicit form stays portable.

Rule: entities *inside* the service transaction, DTOs *across* the boundary.

The N+1 problem and its four fixes



Detect it: count statements in tests (Statistics), read the SQL log in dev, watch p95 latency in prod.

```java
// The problem: 1 query for the orders, then 1 per order for the lines
@Transactional(readOnly = true)
public int linesOfPaidOrders() {
    List<Order> paid = orders.findByStatus(OrderStatus.PAID);   // 1
    int n = 0;
    for (Order o : paid) n += o.getLines().size();   // + N lazy loads
    return n;
}

// Fix 1: fetch join in JPQL (one query, one round trip)
@Query("select distinct o from Order o join fetch o.lines where o.status = :s")
List<Order> findByStatusWithLines(@Param("s") OrderStatus s);

// Fix 2: entity graph on a derived query
@EntityGraph(attributePaths = {"lines", "lines.product"})
List<Order> findByStatus(OrderStatus status);

// Fix 3: batch the lazy loads (IN (?, ?, …)) instead of one per owner
@OneToMany(mappedBy = "order", cascade = CascadeType.ALL,
           orphanRemoval = true)
@BatchSize(size = 32)                 // org.hibernate.annotations
private List<OrderLine> lines = new ArrayList<>();

// Fix 4: do not load entities at all
@Query("select count(l) from OrderLine l where l.order.status = :s")
long countLines(@Param("s") OrderStatus s);
```

⚠ Only one `List` collection can be fetch-joined per query (`MultipleBagFetchException`); use `Set`, or fetch the second collection with a batch size. `distinct` in a fetch join removes duplicate parents in memory (Hibernate 6+ does this automatically and no longer passes DISTINCT to SQL).

Tuning knobs, and how to measure before you turn them

```yaml
# application.yml
spring:
  jpa:
    open-in-view: false        # no EntityManager bound to the web request
    show-sql: false            # prints to stdout; use the logger instead
    properties:
      hibernate:
        default_batch_fetch_size: 32   # IN-lists for lazy associations
        jdbc.batch_size: 50            # group INSERT/UPDATE statements
        order_inserts: true            # sort by table so batches form
        order_updates: true
        generate_statistics: true      # dev and test only
        query.plan_cache_max_size: 2048
logging:
  level:
    org.hibernate.SQL: DEBUG                # the statements
    org.hibernate.orm.jdbc.bind: TRACE      # bound parameter values
    org.hibernate.stat: DEBUG               # per-session statistics
```

```java
// Count statements in a test with Hibernate Statistics
@Autowired EntityManagerFactory emf;

@Test
void paidOrdersReportRunsOneQuery() {
    Statistics stats = emf.unwrap(SessionFactory.class).getStatistics();
    stats.clear();

    report.linesOfPaidOrders();

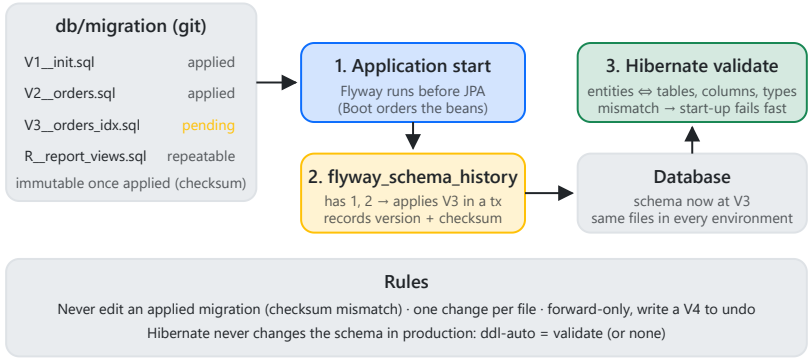
    assertThat(stats.getPrepareStatementCount()).isEqualTo(1);
}
```

Symptom	Lever
Many identical small SELECTs	N+1: fetch join, `@EntityGraph`, `@BatchSize`, projection
1000 INSERTs for one import	`jdbc.batch_size` + `SEQUENCE` ids (IDENTITY disables batching); `StatelessSession`
Slow commit, huge UPDATE with all columns	`@DynamicUpdate`, or split rarely-changed data into another entity
Memory grows during a loop	`flush()` + `clear()` every N items, `Stream` with `readOnly`
Connections held during view rendering	`open-in-view=false`; DTOs out of the service
Same reference data loaded on every request	second-level cache (`@Cacheable`, JCache/Infinispan), or an application cache
Query slow even with 1 statement	indexes (Topic 10), `EXPLAIN`, keyset pagination

- **Open-in-view** keeps a persistence context (and a connection) open for the whole HTTP request so views can lazy-load. Convenient, but it hides N+1 problems and exhausts the pool under load. Spring Boot logs a warning when you leave it at the default.
- `show-sql` is unformatted stdout; the `org.hibernate.SQL` logger goes through SLF4J and can be switched per environment.
- Statistics cost a little CPU: enable them in tests and staging, not in production.

A query-count assertion in the test suite is the cheapest regression guard against N+1.

Schema as versioned code: Flyway migrations




```sql
-- src/main/resources/db/migration/V1__init.sql
CREATE SEQUENCE product_seq START WITH 1 INCREMENT BY 50;

CREATE TABLE product (
    id        BIGINT        PRIMARY KEY,
    sku       VARCHAR(20)   NOT NULL UNIQUE,
    name      VARCHAR(120)  NOT NULL,
    amount    DECIMAL(12,2) NOT NULL,
    currency  CHAR(3)       NOT NULL,
    category  VARCHAR(20)   NOT NULL,
    stock     INT           NOT NULL CHECK (stock >= 0)
);

-- V2__orders.sql
CREATE SEQUENCE orders_seq START WITH 1 INCREMENT BY 50;
CREATE TABLE orders (
    id          BIGINT PRIMARY KEY,
    customer_id BIGINT NOT NULL REFERENCES customer (id),
    status      VARCHAR(20) NOT NULL,
    placed_at   TIMESTAMP WITH TIME ZONE NOT NULL,
    version     BIGINT NOT NULL DEFAULT 0
);

-- V3__orders_idx.sql   (a new file, never edit V1 or V2)
CREATE INDEX idx_orders_customer_placed
    ON orders (customer_id, placed_at DESC);
```

```yaml
spring:
  jpa:
    hibernate:
      ddl-auto: validate          # Hibernate checks, Flyway changes
  flyway:
    locations: classpath:db/migration
    validate-on-migrate: true
```

Schema management in practice: `ddl-auto`, generated DDL, tests

`ddl-auto`	Behaviour	Use
`none`	Hibernate touches nothing	production with Flyway/Liquibase (Boot's default when a migration tool is present)
`validate`	compare mapping with schema, fail on mismatch	everywhere: catches drift at start-up
`update`	add missing tables/columns, never drops or renames	never in production: silent half-migrations
`create`, `create-drop`	drop and recreate from the entities	throwaway prototypes, some tests (Boot's default for embedded DBs without Flyway)

Flyway file	Meaning
`V3__add_index.sql`	versioned: runs once, in order
`R__views.sql`	repeatable: re-runs when its checksum changes
`V3_1__hotfix.sql`	version 3.1: between 3 and 4
Java migration `V4__Backfill.java`	when SQL is not enough (data transformations)

```
// Generate a DDL draft from the entities, then hand-edit it into V1
// (run once with these properties, then remove them)
// spring.jpa.properties.jakarta.persistence.schema-generation
//   .scripts.action=create
//   .scripts.create-target=target/schema.sql

// Test that the migrated schema matches the entities (H2 or a
// Testcontainers PostgreSQL with @ServiceConnection)
@DataJpaTest(properties = {
        "spring.flyway.enabled=true",
        "spring.jpa.hibernate.ddl-auto=validate" })
@AutoConfigureTestDatabase(replace = Replace.NONE)   // keep our datasource
class SchemaMatchesEntitiesTest {

    @Autowired TestEntityManager em;

    @Test
    void productRoundTrip() {
        var p = new Product("KB-1", "Keyboard", Money.eur("49.90"),
                            Category.HARDWARE);
        Long id = em.persistAndGetId(p, Long.class);
        em.flush(); em.clear();

        Product found = em.find(Product.class, id);
        assertThat(found.getPrice()).isEqualTo(Money.eur("49.90"));
    }
}
```

💡 **Liquibase** is the alternative: changelogs in XML/YAML/SQL, rollback support, similar Boot integration (`spring.liquibase.*`). Pick one per project.

⚠ Entity change without a migration = start-up failure with `validate`. That is the point: the schema is reviewed like code.

Common mistakes (and what they cost you)

EAGER everywhere

Every `findById` drags the whole object graph; two eager `List`s explode into a cartesian product. Map LAZY, fetch per use case.

Leaving `open-in-view` on

Lazy loading "works" in controllers and templates, so N+1 goes unnoticed and connections are held for the whole request. Set it to `false` from day one.

Lombok `@Data` / `@ToString` on entities

`toString` and `hashCode` walk associations: recursion, LazyInitializationException, entities vanishing from `Set`s. Write them by hand, by id.

Entities in REST responses

Jackson serialises lazy proxies and bidirectional links: 500 errors, infinite JSON, leaked columns. Return projections or DTOs.

`ddl-auto=update` in production

Adds columns but never drops, renames or migrates data; two developers produce two schemas. Flyway + `validate`.

Modifying outside a transaction

Without `@Transactional` the entity is detached the moment the repository returns; setters change nothing, `@Modifying` throws `TransactionRequiredException`.

`EnumType.ORDINAL` and `IDENTITY` by default

Reordering an enum corrupts stored data; identity columns disable insert batching. Use `STRING` and `SEQUENCE`.

`save()` in a loop, `findAll()` for one row

Each `save` of a detached entity is a SELECT + UPDATE; `findAll().stream().filter(...)` loads the table. Use derived queries, bulk updates and batching.

Ignoring the SQL log

"It works" on 5 test rows and dies on 50 000. Read the statements Hibernate generates and count them in tests.

JPA and Spring Data best practices

1 LAZY by default, fetch per query

Override `@ManyToOne` to LAZY; use `join fetch`, `@EntityGraph` or `@BatchSize` where the use case needs the graph.

2 Transactions at the service layer

One `@Transactional` per use case, `readOnly = true` for queries, `open-in-view=false`. The transaction is the unit of work and the boundary of lazy loading.

3 Aggregate roots own their children

`Order.addLine()` keeps both sides in sync; `cascade = ALL, orphanRemoval = true` only on composition. Shared entities (Product) are referenced, never cascaded.

4 Identity done right

`SEQUENCE` ids with a pooled allocation, `equals` by id when non-null, constant `hashCode`, `toString` without associations, `@Version` on anything users edit concurrently.

5 Value objects as `@Embeddable` records

Money, Address, DateRange: immutable, validated in the constructor, compared by value, mapped to plain columns.

6 Projections across the boundary

Entities inside the transaction, records or interface projections out to controllers. Paginate with `Slice` or keyset when totals are not needed.

7 Let the database do bulk work

`@Modifying` updates, aggregates in JPQL, `StatelessSession` for imports, JDBC batching switched on. Do not loop over entities to sum a column.

8 Versioned schema, validated mapping

Flyway migrations reviewed with the code, `ddl-auto=validate`, indexes for every foreign key and every `WHERE`/`ORDER BY` you paginate on.

9 Measure: SQL log and query counts

Log SQL with bound parameters in dev, assert statement counts in tests, enable statistics in staging. Know the SQL your mapping produces.

Chapter quiz 10 questions

1. Which identifier generation strategy prevents Hibernate from batching INSERT statements?

- SEQUENCE with allocationSize = 50
- IDENTITY
- UUID
- AUTO

2. In a bidirectional one-to-many, what does `mappedBy = "order"` on `Order.lines` declare?

- Order owns the foreign key column
- The collection is fetched eagerly
- OrderLine.order is the owning side and holds the FK column
- A join table must be created

3. What are the default fetch types of `@ManyToOne` and `@OneToMany`, respectively?

- LAZY / LAZY
- EAGER / LAZY
- EAGER / EAGER
- LAZY / EAGER

4. You touch a lazy collection after the transaction ended (`open-in-view=false`). What happens?

- Hibernate opens a new connection and loads it
- An empty list is returned
- A LazyInitializationException is thrown
- It is served from the second-level cache

5. How does Hibernate know it must issue an UPDATE for a managed entity you modified without calling `save()`?

- The setter sends SQL immediately
- Dirty checking: at flush it compares the entity with the snapshot taken when it was loaded
- Spring Data intercepts setters with a proxy
- It re-reads the row and diffs it

6. Two transactions load Order 42 with version 3 and both update it. What happens to the second commit?

- It silently overwrites the first update
- It waits for a row lock
- It fails with an OptimisticLockException because UPDATE … WHERE version = 3 affects 0 rows
- Hibernate merges both changes

7. Which return type avoids the extra count query when paginating?

- Page<T>
- List<T> with Pageable
- Slice<T>
- Stream<T>

8. Which of these does NOT fix an N+1 problem?

- join fetch in JPQL
- @EntityGraph(attributePaths = "lines")
- @BatchSize / default_batch_fetch_size
- spring.jpa.open-in-view=true

9. With Flyway managing the schema, which `spring.jpa.hibernate.ddl-auto` value should production use?

- update
- create-drop
- validate
- create

10. Why is generating `equals`/`hashCode` from all entity fields (Lombok `@Data`) a mistake?

- It is slower than the default
- The hash changes when the id is assigned or a field is edited, so the entity is lost in a HashSet and associations may be loaded
- JPA forbids overriding equals
- It disables dirty checking

WRAP-UP

Summary

JPA is the specification, Hibernate 7.4 the provider, Spring Data JPA the repository layer: entities and interfaces replace hand-written SQL for CRUD.

Entities need an `@Id` (prefer `SEQUENCE`), a no-arg constructor, `STRING` enums and `@Embeddable` records for value objects.

The persistence context is a first-level cache with snapshots: one instance per id, changes flushed at commit (dirty checking).

Associations map to foreign keys; the owning side writes the column, `mappedBy` mirrors it; helpers keep both sides in sync.

Map LAZY, cascade only on composition, use `orphanRemoval` for children; fetch what a use case needs with `join fetch` or `@EntityGraph`.

Derived queries, `@Query` JPQL, native SQL, Criteria/`Specification`: pick the simplest that expresses the query; project into records for reads.

`@Transactional` services are the unit of work; `@Version` catches lost updates, `@Lock(PESSIMISTIC_WRITE)` serialises hot rows.

`Page` counts, `Slice` does not, keyset scrolling skips OFFSET; N+1 is found by counting statements and fixed by fetching smarter.

Flyway owns the schema, Hibernate validates it (`ddl-auto=validate`), and `open-in-view=false` keeps persistence out of the web layer.

Open Lab-03: 5 tasks + 1 challenge

Next chapter: 04 · RESTful Web Services (JAX-RS)